compact2, compact3
org.w3c.dom

Interface Element

All Superinterfaces:

Node

All Known Subinterfaces:

Detail, DetailEntry, SOAPBody, SOAPBodyElement, SOAPElement, SOAPEnvelope, SOAPFault, SOAPFaultElement, SOAPHeader, SOAPHeaderElement

All Known Implementing Classes:

IIOMetadataNode

```
public interface Element
extends Node
```

The `Element` interface represents an element in an HTML or XML document. Elements may have attributes associated with them; since the `Element` interface inherits from `Node`, the generic `Node` interface attribute `attributes` may be used to retrieve the set of all attributes for an element. There are methods on the `Element` interface to retrieve either an `Attr` object by name or an attribute value by name. In XML, where an attribute value may contain entity references, an `Attr` object should be retrieved to examine the possibly fairly complex sub-tree representing the attribute value. On the other hand, in HTML, where all attributes have simple string values, methods to directly access an attribute value can safely be used as a convenience.

Note: In DOM Level 2, the method `normalize` is inherited from the `Node` interface where it was moved.

See also the Document Object Model (DOM) Level 3 Core Specification.

Field Summary

Fields inherited from interface org.w3c.dom.Node

ATTRIBUTE_NODE, CDATA_SECTION_NODE, COMMENT_NODE, DOCUMENT_FRAGMENT_NODE, DOCUMENT_NODE, DOCUMENT_POSITION_CONTAINED_BY, DOCUMENT_POSITION_CONTAINS, DOCUMENT_POSITION_DISCONNECTED, DOCUMENT_POSITION_FOLLOWING, DOCUMENT_POSITION_IMPLEMENTATION_SPECIFIC, DOCUMENT_POSITION_PRECEDING, DOCUMENT_TYPE_NODE, ELEMENT_NODE, ENTITY_NODE, ENTITY_REFERENCE_NODE, NOTATION_NODE, PROCESSING_INSTRUCTION_NODE, TEXT_NODE

Method Summary

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method	Description
String	getAttribute(String name)	Retrieves an attribute value by name.
Attr	getAttributeNode(String name)	Retrieves an attribute node by name.
Attr	getAttributeNodeNS(String namespaceURI, String localName)	Retrieves an `Attr` node by local name and namespace URI.
String	getAttributeNS(String namespaceURI, String localName)	Retrieves an attribute value by local name and namespace URI.
NodeList	getElementsByTagName(String name)	Returns a `NodeList` of all descendant `Elements` with a given tag name, in document order.
NodeList	getElementsByTagNameNS(String namespaceURI, String localName)	Returns a `NodeList` of all the descendant `Elements` with a given local name and namespace URI in document order.
TypeInfo	getSchemaTypeInfo()	The type information associated with this element.
String	getTagName()	The name of the element.
boolean	hasAttribute(String name)	Returns `true` when an attribute with a given name is specified on this element or has a default value, `false` otherwise.
boolean	hasAttributeNS(String namespaceURI, String localName)	Returns `true` when an attribute with a given local name and namespace URI is specified on this element or has a default value, `false` otherwise.
void	removeAttribute(String name)	Removes an attribute by name.
Attr	removeAttributeNode(Attr oldAttr)	Removes the specified attribute node.
void	removeAttributeNS(String namespaceURI, String localName)	Removes an attribute by local name and namespace URI.
void	setAttribute(String name, String value)	Adds a new attribute.
Attr	setAttributeNode(Attr newAttr)	Adds a new attribute node.
Attr	setAttributeNodeNS(Attr newAttr)	Adds a new attribute.
void	setAttributeNS(String namespaceURI, String qualifiedName, String value)	Adds a new attribute.
void	setIdAttribute(String name, boolean isId)	If the parameter `isId` is `true`, this method declares the specified attribute to be a user-determined ID attribute .
void	setIdAttributeNode(Attr idAttr, boolean isId)	If the parameter `isId` is `true`, this method declares the specified attribute to be a user-determined ID attribute .
void	setIdAttributeNS(String namespaceURI, String localName, boolean isId)	If the parameter `isId` is `true`, this method declares the specified attribute to be a user-determined ID attribute .

Methods inherited from interface org.w3c.dom.Node

appendChild, cloneNode, compareDocumentPosition, getAttributes, getBaseURI, getChildNodes, getFeature, getFirstChild, getLastChild, getLocalName, getNamespaceURI, getNextSibling, getNodeName, getNodeType, getNodeValue, getOwnerDocument, getParentNode, getPrefix, getPreviousSibling, getTextContent, getUserData, hasAttributes, hasChildNodes, insertBefore, isDefaultNamespace, isEqualNode, isSameNode, isSupported, lookupNamespaceURI, lookupPrefix, normalize, removeChild, replaceChild, setNodeValue, setPrefix, setTextContent, setUserData

Method Detail

getTagName

```
String getTagName()
```

The name of the element. If `Node.localName` is different from `null`, this attribute is a qualified name. For example, in:

```
 <elementExample id="demo"> ...
 </elementExample> ,
```

`tagName` has the value `"elementExample"`. Note that this is case-preserving in XML, as are all of the operations of the DOM. The HTML DOM returns the `tagName` of an HTML element in the canonical uppercase form, regardless of the case in the source HTML document.

getAttribute

```
String getAttribute(String name)
```

Retrieves an attribute value by name.

Parameters:

`name` - The name of the attribute to retrieve.

Returns:

The Attr value as a string, or the empty string if that attribute does not have a specified or default value.

setAttribute

```
void setAttribute(String name,
                  String value)
           throws DOMException
```

Adds a new attribute. If an attribute with that name is already present in the element, its value is changed to be that of the value parameter. This value is a simple string; it is not parsed as it is being set. So any markup (such as syntax to be recognized as an entity reference) is treated as literal text, and needs to be appropriately escaped by the implementation when it is written out. In order to assign an attribute value that contains entity references, the user must create an `Attr` node plus any `Text` and `EntityReference` nodes, build the appropriate subtree, and use `setAttributeNode` to assign it as the value of an attribute.
To set an attribute with a qualified name and namespace URI, use the `setAttributeNS` method.

Parameters:

`name` - The name of the attribute to create or alter.

`value` - Value to set in string form.

Throws:

`DOMException` - INVALID_CHARACTER_ERR: Raised if the specified name is not an XML name according to the XML version in use specified in the Document.xmlVersion attribute.
NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.

removeAttribute

```
void removeAttribute(String name)
             throws DOMException
```

Removes an attribute by name. If a default value for the removed attribute is defined in the DTD, a new attribute immediately appears with the default value as well as the corresponding namespace URI, local name, and prefix when applicable. The implementation may handle default values from other schemas similarly but applications should use `Document.normalizeDocument()` to guarantee this information is up-to-date.
If no attribute with this name is found, this method has no effect.
To remove an attribute by local name and namespace URI, use the `removeAttributeNS` method.

Parameters:

`name` - The name of the attribute to remove.

Throws:

`DOMException` - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.

getAttributeNode

```
Attr getAttributeNode(String name)
```

Retrieves an attribute node by name.
To retrieve an attribute node by qualified name and namespace URI, use the `getAttributeNodeNS` method.

Parameters:

`name` - The name (nodeName) of the attribute to retrieve.

Returns:

The Attr node with the specified name (nodeName) or null if there is no such attribute.

setAttributeNode

```
Attr setAttributeNode(Attr newAttr)
              throws DOMException
```

Adds a new attribute node. If an attribute with that name (`nodeName`) is already present in the element, it is replaced by the new one. Replacing an attribute node by itself has no effect.
To add a new attribute node with a qualified name and namespace URI, use the `setAttributeNodeNS` method.

Parameters:

`newAttr` - The Attr node to add to the attribute list.

Returns:

If the newAttr attribute replaces an existing attribute, the replaced Attr node is returned, otherwise null is returned.

Throws:

`DOMException` - WRONG_DOCUMENT_ERR: Raised if newAttr was created from a different document than the one that created the element.
NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
INUSE_ATTRIBUTE_ERR: Raised if newAttr is already an attribute of another Element object. The DOM user must explicitly clone Attr nodes to re-use them in other elements.

removeAttributeNode

```
Attr removeAttributeNode(Attr oldAttr)
```

```
Attr removeAttributeNode(Attr oldAttr)
                  throws DOMException
```

Removes the specified attribute node. If a default value for the removed `Attr` node is defined in the DTD, a new node immediately appears with the default value as well as the corresponding namespace URI, local name, and prefix when applicable. The implementation may handle default values from other schemas similarly but applications should use `Document.normalizeDocument()` to guarantee this information is up-to-date.

Parameters:

`oldAttr` - The Attr node to remove from the attribute list.

Returns:

The Attr node that was removed.

Throws:

`DOMException` - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
NOT_FOUND_ERR: Raised if oldAttr is not an attribute of the element.

getElementsByTagName

```
NodeList getElementsByTagName(String name)
```

Returns a `NodeList` of all descendant `Elements` with a given tag name, in document order.

Parameters:

`name` - The name of the tag to match on. The special value "*" matches all tags.

Returns:

A list of matching Element nodes.

getAttributeNS

```
String getAttributeNS(String namespaceURI,
                      String localName)
               throws DOMException
```

Retrieves an attribute value by local name and namespace URI.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

`namespaceURI` - The namespace URI of the attribute to retrieve.

`localName` - The local name of the attribute to retrieve.

Returns:

The Attr value as a string, or the empty string if that attribute does not have a specified or default value.

Throws:

`DOMException` - NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

setAttributeNS

```
void setAttributeNS(String namespaceURI,
                    String qualifiedName,
                    String value)
             throws DOMException
```

Adds a new attribute. If an attribute with the same local name and namespace URI is already present on the element, its prefix is changed to be the prefix part of the `qualifiedName`, and its value is changed to be the `value` parameter. This value is a simple string; it is not parsed as it is being set. So any markup (such as syntax to be recognized as an entity reference) is treated as literal text, and needs to be appropriately escaped by the implementation when it is written out. In order to assign an attribute value that contains entity references, the user must create an `Attr` node plus any `Text` and `EntityReference` nodes, build the appropriate subtree, and use `setAttributeNodeNS` or `setAttributeNode` to assign it as the value of an attribute.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

`namespaceURI` - The namespace URI of the attribute to create or alter.

`qualifiedName` - The qualified name of the attribute to create or alter.

`value` - The value to set in string form.

Throws:

`DOMException` - INVALID_CHARACTER_ERR: Raised if the specified qualified name is not an XML name according to the XML version in use specified in the Document.xmlVersion attribute.
NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
NAMESPACE_ERR: Raised if the qualifiedName is malformed per the Namespaces in XML specification, if the qualifiedName has a prefix and the namespaceURI is null, if the qualifiedName has a prefix that is "xml" and the namespaceURI is different from " http://www.w3.org/XML/1998/namespace", if the qualifiedName or its prefix is "xmlns" and the namespaceURI is different from "http://www.w3.org/2000/xmlns/", or if the namespaceURI is "http://www.w3.org/2000/xmlns/" and neither the qualifiedName nor its prefix is "xmlns".
NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

removeAttributeNS

```
void removeAttributeNS(String namespaceURI,
                       String localName)
                throws DOMException
```

Removes an attribute by local name and namespace URI. If a default value for the removed attribute is defined in the DTD, a new attribute immediately appears with the default value as well as the corresponding namespace URI, local name, and prefix when applicable. The implementation may handle default values from other schemas similarly but applications should use `Document.normalizeDocument()` to guarantee this information is up-to-date.
If no attribute with this local name and namespace URI is found, this method has no effect.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

`namespaceURI` - The namespace URI of the attribute to remove.

`localName` - The local name of the attribute to remove.

Throws:

`DOMException` - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.

NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

getAttributeNodeNS

```
Attr getAttributeNodeNS(String namespaceURI,
                        String localName)
                 throws DOMException
```

Retrieves an `Attr` node by local name and namespace URI.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

`namespaceURI` - The namespace URI of the attribute to retrieve.

`localName` - The local name of the attribute to retrieve.

Returns:

The Attr node with the specified attribute local name and namespace URI or null if there is no such attribute.

Throws:

`DOMException` - NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

setAttributeNodeNS

```
Attr setAttributeNodeNS(Attr newAttr)
                 throws DOMException
```

Adds a new attribute. If an attribute with that local name and that namespace URI is already present in the element, it is replaced by the new one. Replacing an attribute node by itself has no effect.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

`newAttr` - The Attr node to add to the attribute list.

Returns:

If the newAttr attribute replaces an existing attribute with the same local name and namespace URI, the replaced Attr node is returned, otherwise null is returned.

Throws:

`DOMException` - WRONG_DOCUMENT_ERR: Raised if newAttr was created from a different document than the one that created the element.
NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
INUSE_ATTRIBUTE_ERR: Raised if newAttr is already an attribute of another Element object. The DOM user must explicitly clone Attr nodes to re-use them in other elements.
NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

getElementsByTagNameNS

```
NodeList getElementsByTagNameNS(String namespaceURI,
                                String localName)
                         throws DOMException
```

Returns a `NodeList` of all the descendant `Elements` with a given local name and namespace URI in document order.

Parameters:

`namespaceURI` - The namespace URI of the elements to match on. The special value "*" matches all namespaces.

`localName` - The local name of the elements to match on. The special value "*" matches all local names.

Returns:

A new NodeList object containing all the matched Elements.

Throws:

`DOMException` - NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

hasAttribute

```
boolean hasAttribute(String name)
```

Returns `true` when an attribute with a given name is specified on this element or has a default value, `false` otherwise.

Parameters:

`name` - The name of the attribute to look for.

Returns:

true if an attribute with the given name is specified on this element or has a default value, false otherwise.

Since:

DOM Level 2

hasAttributeNS

```
boolean hasAttributeNS(String namespaceURI,
                       String localName)
                throws DOMException
```

Returns `true` when an attribute with a given local name and namespace URI is specified on this element or has a default value, `false` otherwise.
Per [XML Namespaces] , applications must use the value `null` as the `namespaceURI` parameter for methods if they wish to have no namespace.

Parameters:

namespaceURI - The namespace URI of the attribute to look for.

localName - The local name of the attribute to look for.

Returns:

true if an attribute with the given local name and namespace URI is specified or has a default value on this element, false otherwise.

Throws:

DOMException - NOT_SUPPORTED_ERR: May be raised if the implementation does not support the feature "XML" and the language exposed through the Document does not support XML Namespaces (such as [HTML 4.01]).

Since:

DOM Level 2

getSchemaTypeInfo

```
TypeInfo getSchemaTypeInfo()
```

The type information associated with this element.

Since:

DOM Level 3

setIdAttribute

```
void setIdAttribute(String name,
                    boolean isId)
             throws DOMException
```

If the parameter isId is true, this method declares the specified attribute to be a user-determined ID attribute . This affects the value of Attr.isId and the behavior of Document.getElementById, but does not change any schema that may be in use, in particular this does not affect the Attr.schemaTypeInfo of the specified Attr node. Use the value false for the parameter isId to undeclare an attribute for being a user-determined ID attribute.
To specify an attribute by local name and namespace URI, use the setIdAttributeNS method.

Parameters:

name - The name of the attribute.

isId - Whether the attribute is a of type ID.

Throws:

DOMException - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
NOT_FOUND_ERR: Raised if the specified node is not an attribute of this element.

Since:

DOM Level 3

setIdAttributeNS

```
void setIdAttributeNS(String namespaceURI,
                      String localName,
                      boolean isId)
               throws DOMException
```

If the parameter isId is true, this method declares the specified attribute to be a user-determined ID attribute . This affects the value of Attr.isId and the behavior of Document.getElementById, but does not change any schema that may be in use, in particular this does not affect the Attr.schemaTypeInfo of the specified Attr node. Use the value false for the parameter isId to undeclare an attribute for being a user-determined ID attribute.

Parameters:

namespaceURI - The namespace URI of the attribute.

localName - The local name of the attribute.

isId - Whether the attribute is a of type ID.

Throws:

DOMException - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
NOT_FOUND_ERR: Raised if the specified node is not an attribute of this element.

Since:

DOM Level 3

setIdAttributeNode

```
void setIdAttributeNode(Attr idAttr,
                        boolean isId)
                 throws DOMException
```

If the parameter isId is true, this method declares the specified attribute to be a user-determined ID attribute . This affects the value of Attr.isId and the behavior of Document.getElementById, but does not change any schema that may be in use, in particular this does not affect the Attr.schemaTypeInfo of the specified Attr node. Use the value false for the parameter isId to undeclare an attribute for being a user-determined ID attribute.

Parameters:

idAttr - The attribute node.

isId - Whether the attribute is a of type ID.

Throws:

DOMException - NO_MODIFICATION_ALLOWED_ERR: Raised if this node is readonly.
NOT_FOUND_ERR: Raised if the specified node is not an attribute of this element.

Since:

DOM Level 3

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2025, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy. Modify Cookie Preferences. Modify Ad Choices.